ERC-20 Token Creation and Deployment using Remix IDE and MetaMask

Name: Ved V. Joshi · Roll No: 46 · Class: D20A

Aim

To create an ERC-20 token using Solidity in the Remix IDE and deploy the token contract to a selected testnet (Sepolia) using a MetaMask wallet. The experiment also studies the basic ERC-20 attributes, functions and events.

Theory

ERC-20 is the standard interface for fungible tokens on Ethereum-compatible blockchains. Every unit of an ERC-20 token is interchangeable with every other unit, so wallets, exchanges and DApps can handle any compliant token the same way. A token is a smart contract that stores each address's balance and exposes a fixed set of attributes, functions and events.

Member	Kind	Purpose
`name`, `symbol`	Attribute	Human-readable token name and ticker (here "Lab Token" and "LAB")
`decimals`	Attribute	Number of decimal places; 18 means 1 token = 10^18 smallest units
`totalSupply`	Attribute	Total number of units in existence
`balanceOf(owner)`	Function	Returns the token balance of an address
`transfer(to, amount)`	Function	Moves tokens from the caller to another address
`approve(spender, amount)`	Function	Lets a spender use up to `amount` of the caller's tokens
`allowance(owner, spender)`	Function	Returns how much a spender may still use

Member	Kind	Purpose
`transferFrom(from, to, amount)`	Function	Moves tokens on behalf of the owner, using the approved allowance
`Transfer`	Event	Logged on every movement of tokens, including the first mint
`Approval`	Event	Logged whenever an allowance is set

Solidity is the language used to write the contract. The Remix IDE is a browser-based environment to write, compile and deploy it. MetaMask is the browser wallet that holds the deployer's account and signs the deployment and every later transaction after the user confirms it. Remix connects to MetaMask through the "Injected Provider" environment.

Sepolia is an Ethereum test network. It works like the main network, but its Sepolia ETH has no real value, so contracts can be tried safely. Deploying a contract and every state-changing call, such as `transfer` and `approve`, costs gas paid in Sepolia ETH. Read-only calls such as `name`, `symbol`, `totalSupply` and `balanceOf` are free. A call that breaks a rule in the contract, for example sending more tokens than the sender owns, reverts with an error and changes nothing.

Steps

1. Open the Remix IDE at remix.ethereum.org, create a file named `MyToken.sol` and write the ERC-20 contract: the `name`, `symbol` and `decimals` attributes, `totalSupply`, the `balanceOf` and `allowance` mappings, the `Transfer` and `Approval` events, and the `transfer`, `approve` and `transferFrom` functions. The constructor mints the initial supply to the deployer.
2. Compile the contract with Solidity 0.8.20 or newer and confirm that Remix shows it as Compiled (Fig. 1).
3. In Deploy & Run Transactions, set the environment to Browser Extension (MetaMask) with the Sepolia test network selected, and choose the MetaMask account that will deploy the token.
4. Enter `1000000` as `initialSupply`, click Deploy, and confirm the "Deploy a contract" request in MetaMask (Fig. 2).
5. Check the deployment transaction and the contract verification output in the Remix console (Fig. 3).

6. Under Deployed Contracts, call the read functions `name`, `symbol` and `totalSupply` to check the token attributes (Fig. 4).
7. Call `transfer` to send 1 token (1000000000000000000 units) to a second MetaMask account and confirm it in MetaMask (Fig. 5). Open the transaction in the console to see its status and logged event (Fig. 6).
8. As a negative test, try a transfer from an account that holds no tokens and note the warning MetaMask shows (Fig. 7).

Output

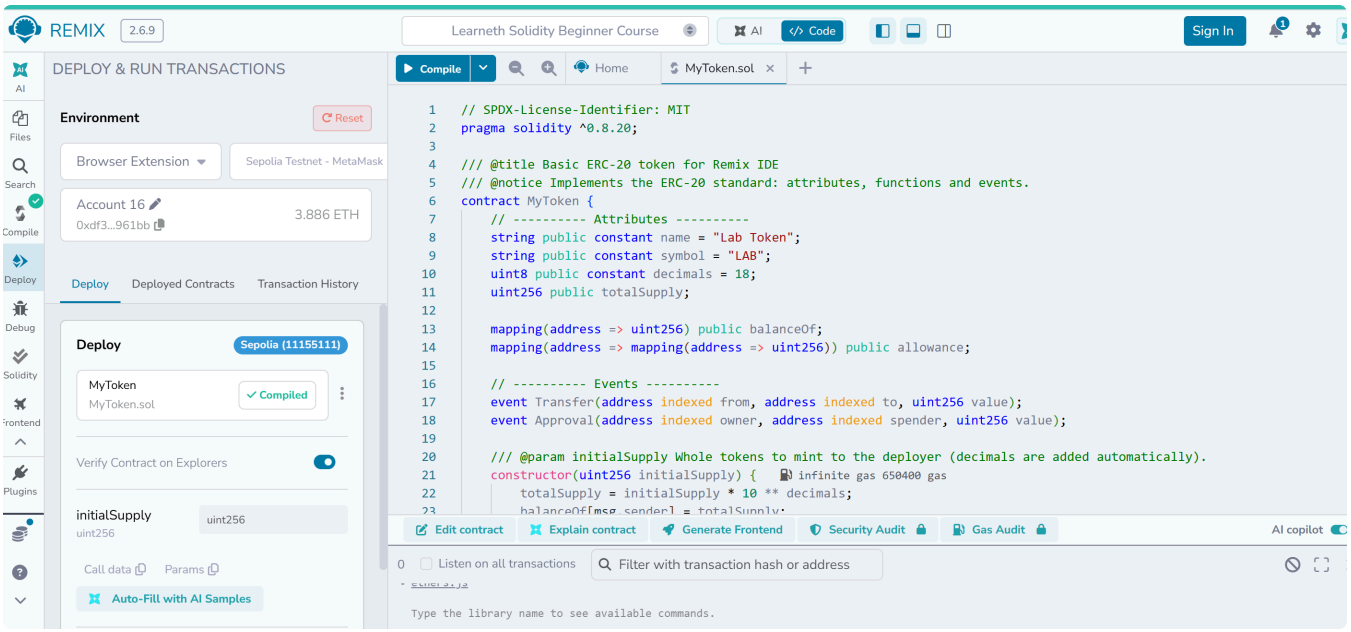


Fig. 1. The ERC-20 contract `MyToken.sol` in Remix, shown as Compiled, with the Sepolia (11155111) network and the deploying MetaMask account selected.

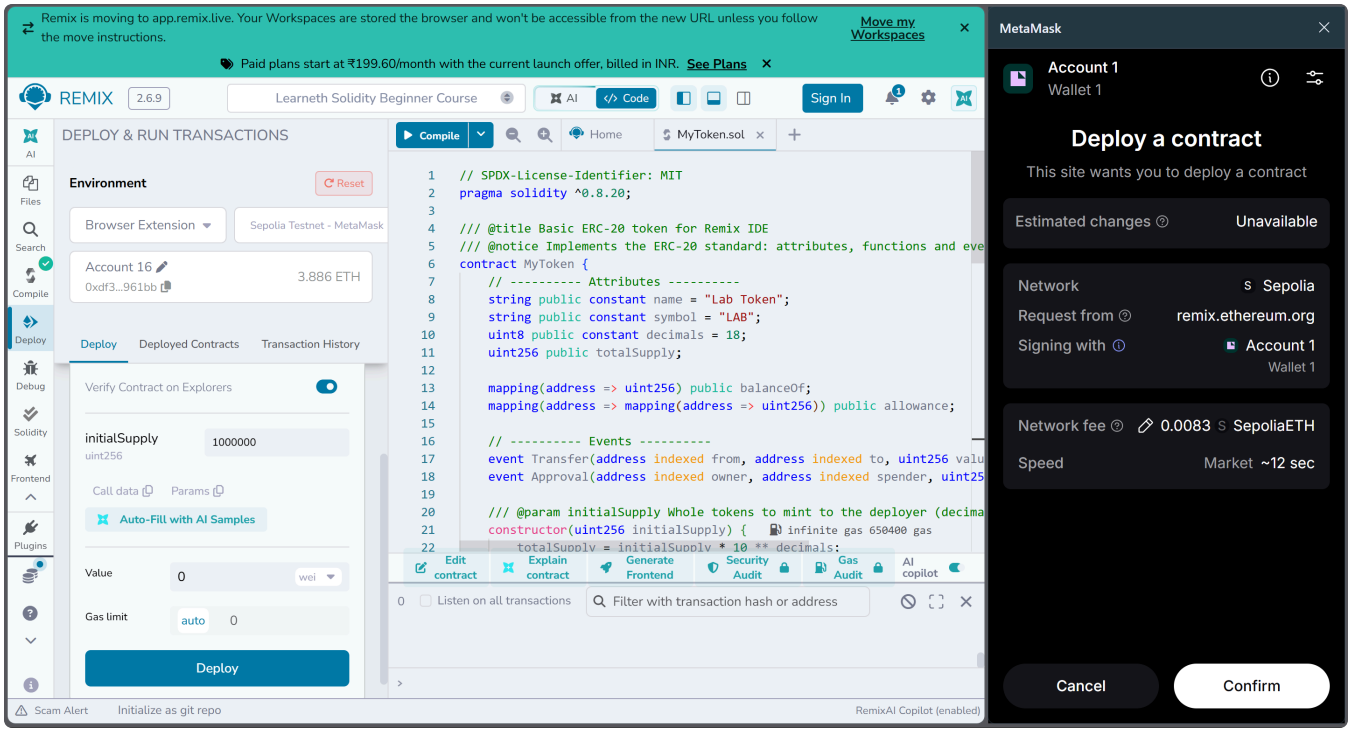


Fig. 2. Deployment with `initialSupply` set to 1000000 and the MetaMask "Deploy a contract" request on the Sepolia network.



Fig. 3. Successful deployment transaction (block 12012467, constructor call, 1 log) and the Sourcify verification result in the Remix console.

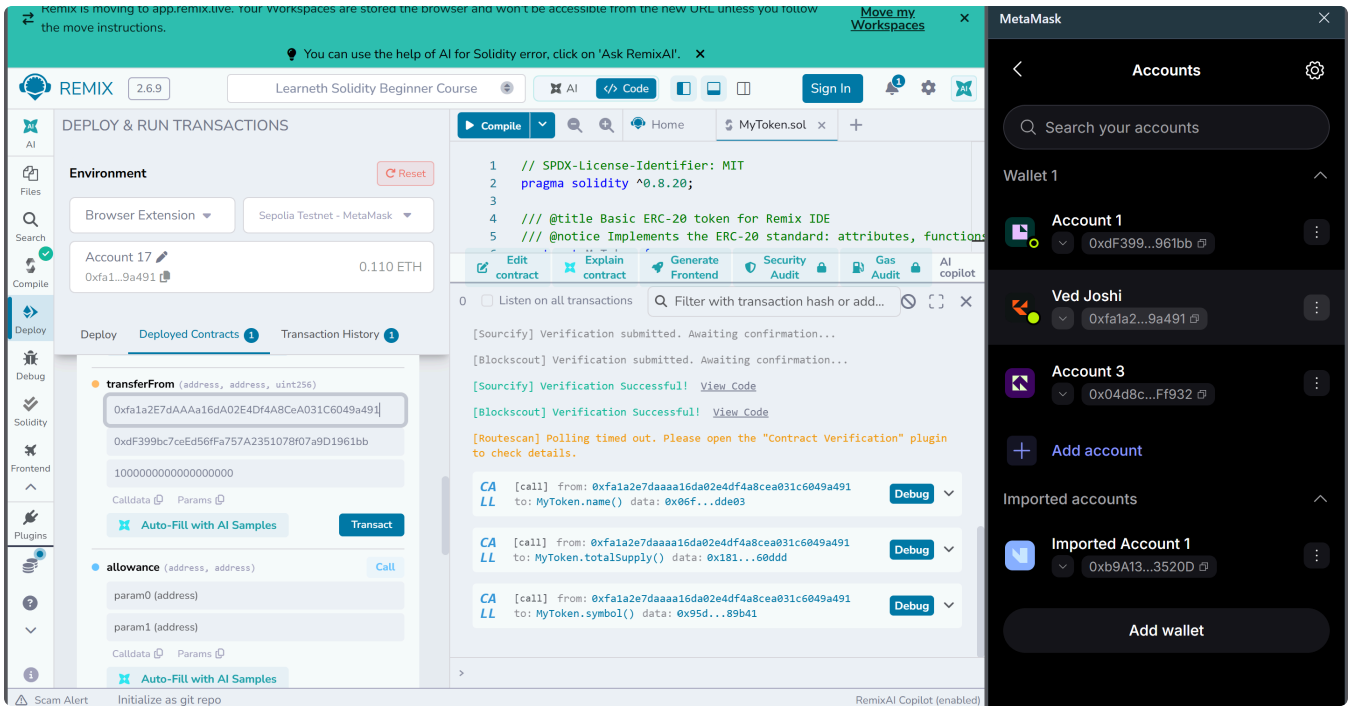


Fig. 4. The deployed MyToken contract with calls to `name()`, `totalSupply()` and `symbol()` in the console, and the MetaMask accounts used for testing.

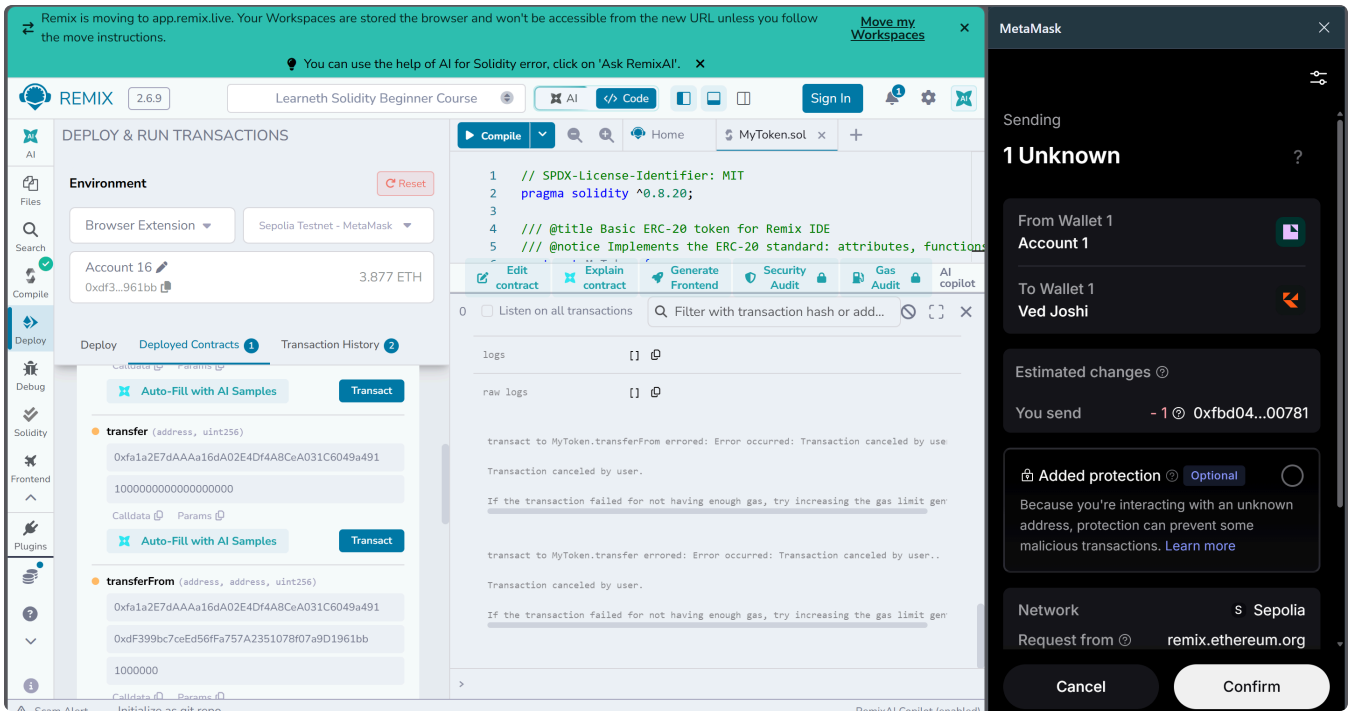


Fig. 5. MetaMask confirmation for sending 1 token from Account 1 to the second account on Sepolia.

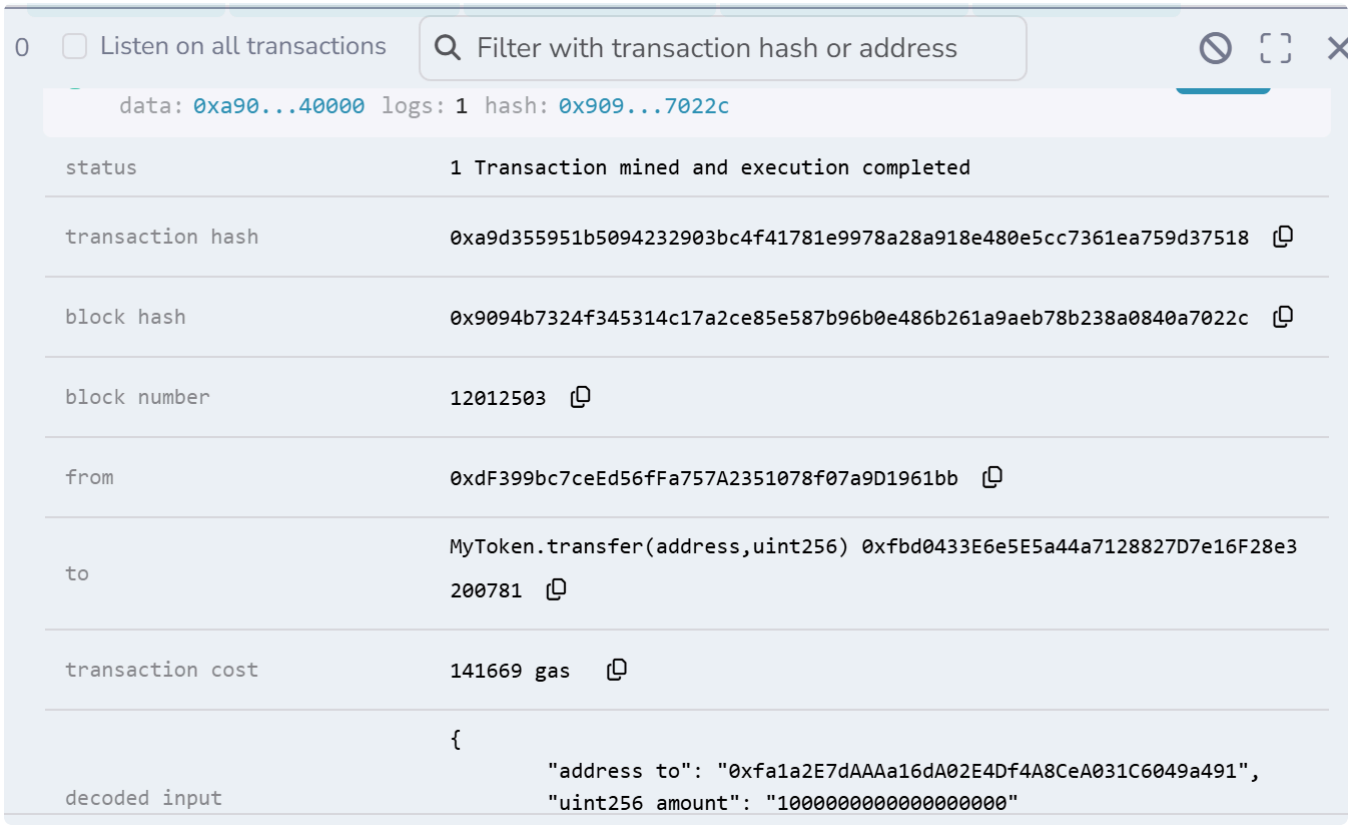


Fig. 6. The mined `transfer` transaction (block 12012503) with the decoded input: 10000000000000000000 units (1 token) sent to the second account, and 1 log, the `Transfer` event.

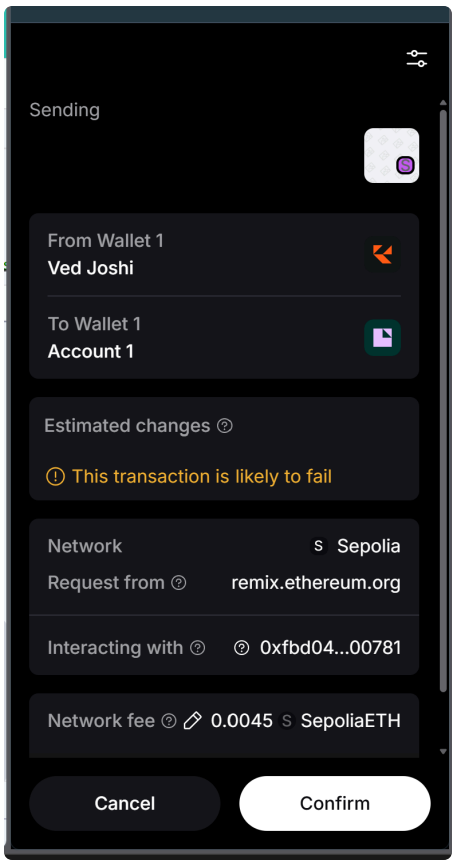


Fig. 7. MetaMask warning "This transaction is likely to fail" when an account that holds no tokens tries to send them.

Conclusion

An ERC-20 token, Lab Token (LAB), with 18 decimals and an initial supply of 1,000,000 tokens, was written in Solidity, compiled in the Remix IDE and deployed to the Sepolia test network through MetaMask. The deployment transaction succeeded, and the contract's `name`, `symbol` and `totalSupply` attributes were read from the deployed contract. A `transfer` of 1 token to a second account was confirmed in MetaMask, mined on Sepolia and logged a `Transfer` event. A transfer attempted from an account with no tokens was flagged by MetaMask as likely to fail, which showed that the contract rejects transfers that exceed the sender's balance. The experiment showed how the standard ERC-20 attributes, functions and events let a wallet and a development environment deploy and use a token without any custom integration.